

PROYECTO NIFI – 1ª EVALUACIÓN

Explicación de la fuente de datos pública utilizada

Para realizar esta práctica, he utilizado la API de la NASA. Concretamente, el apartado "APOD" (*Astronomy Picture of the Day*), que recopila las imágenes del día relacionadas con el espacio, ya sean tomadas por la NASA o por personas particulares.

He decidido utilizar esta fuente de datos porque me gustan los temas relacionados con el espacio y porque me parecía interesante poder obtener imágenes del espacio junto a información adicional de cada una, debido a que luego se podrían usar para *Machine Learning* o *Deep Learning*, por ejemplo. También he decidido utilizar esta API porque la documentación es de calidad, lo que me ha facilitado entender cómo utilizarla.

Descripción del flujo de datos (pasos seguidos)

Antes de comenzar, me gustaría mencionar que este proyecto se divide en dos ramas de flujo de datos:

- Una obtiene y guarda los enlaces a las imágenes.
- Y la otra guarda el enlace de cada imagen junto con su información, conservando el formato JSON. Eso sí, realizando una limpieza de las etiquetas innecesarias.

A continuación, se explicará de manera más detallada como se realiza cada acción.

Inicio:

1. He creado un grupo de procesadores y le he llamado "ProyectoNifi_1Eval".
2. He añadido el procesador "InvokeHTTP" y le he cambiado el nombre a uno más descriptivo. Lo he configurado para que llame a la API cada hora y le he añadido la URL para obtener un JSON, en el cual se encuentran los enlaces a las imágenes espaciales, además de información de cada una de ellas.

En este caso he decidido recopilar 10 imágenes (sin ningún intervalo de fecha) cada vez que se realiza una llamada, aunque, dependiendo del motivo para que se necesiten, se podría aumentar la cantidad que se obtienen en cada petición y, además, agregar un intervalo de fechas.

3. He añadido el procesador "ConvertRecord" y le he cambiado el nombre a uno más descriptivo. En "Record Reader" le he añadido la propiedad "JsonTreeReader" y en "Record Writer" le he añadido la propiedad "JsonRecordSetWriter" (la cual he configurado para que saque el JSON ya formateado, utilizando la propiedad "Pretty Print JSON" en "true"). Esto lo he hecho porque el JSON proveniente de la NASA viene sin formatear, es decir, que todo está en una sola línea, por lo que, si no lo formatease, luego los procesadores "SplitText" y "SplitJson" no funcionarían correctamente.

Primer flujo:

4. He añadido el procesador "SplitText" y le he cambiado el nombre a uno más descriptivo. He configurado la propiedad "Line Split Count" en "1" para que separe cada línea del JSON.
5. He añadido el procesador "RouteOnContent" y le he cambiado el nombre a uno más descriptivo. Antes de explicar cómo lo he configurado, quiero decir que estuve probando con otros procesadores que sí habíamos dado en clase, pero ninguno me funcionaba correctamente, así que investigué y encontré este.
6. Ahora voy explicar cómo he configurado el procesador del paso anterior. La propiedad "Match Requirement" la he configurado en "content must contain match", la propiedad "Content Buffer Size" en "10MB" y, además, le he añadido una propiedad llamada "hasImg" con valor "hdurl". Esta última lo que hace es comprobar si la línea contiene un enlace a una imagen; si la tiene le deja continuar al siguiente procesador y, si no, no.
7. He añadido el procesador "ReplaceText" y le he cambiado el nombre a uno más descriptivo. La propiedad "Maximum Buffer Size" la he configurado en "10MB" y a la propiedad "Search Value" le he añadido el valor `".*\"hdurl\" ?:(http[^\"]*)\".*\"` para que elimine todo lo que no sea parte del enlace.
8. He añadido el procesador "UpdateAttribute" y le he cambiado el nombre a uno más descriptivo. He añadido la propiedad "filename" y le he añadido el valor `"enlaceImagenNasa-#{nextInt()}.txt"` para que le asigne un nombre único a cada imagen y, además, para que estén enumeradas.

Segundo flujo:

9. He añadido el procesador "SplitJson" y le he cambiado el nombre a uno más descriptivo. He configurado la propiedad "JsonPath Expression" añadiéndole el valor `"$[*]"`, para que separe cada elemento (imagen con su información) del JSON en un archivo.

10. He añadido el procesador "JoltTransformJSON" y le he cambiado el nombre a uno más descriptivo. He configurado las propiedades:

- "Jolt Transformation DSL" a "Shift".
- "Pretty Print" a "true".
- Y a "Jolt Specification" le he añadido el siguiente valor para solamente quedarnos con las etiquetas que he considerado de importancia. "Date" (fecha en la que fue tomada la imagen), "title" (título de la imagen), "explanation" (explicación de la imagen) y "hdurl" (enlace que nos lleva a la imagen).

```
1 {  
2   "date": "date",  
3   "title": "title",  
4   "explanation": "explanation",  
5   "hdurl": "hdurl"  
6 }
```

11. He añadido el procesador "UpdateAttribute" y le he cambiado el nombre a uno más descriptivo. He creado la propiedad "filename" y le he añadido el valor "enlaceConInfoNasa- $\${nextInt()}.json$ " para que le asigne un nombre único a cada imagen y, además, para que estén enumeradas

Final:

12. El último procesador que he añadido ha sido "PutFile" y le he cambiado el nombre a uno más descriptivo. Lo he configurado para que su directorio de salida sea la carpeta "/tmp/out".

13. He relacionado todos los procesadores y he configurado las relaciones.

14. Por último, he añadido un *Label* con mi nombre.

Descripción del destino donde se almacenan los datos procesados

Los datos procesados se almacenan en la carpeta "/tmp/out" de NiFi, aunque, en el caso de ser necesario, se podría modificar esta ruta para almacenar los datos en otra carpeta más conveniente.